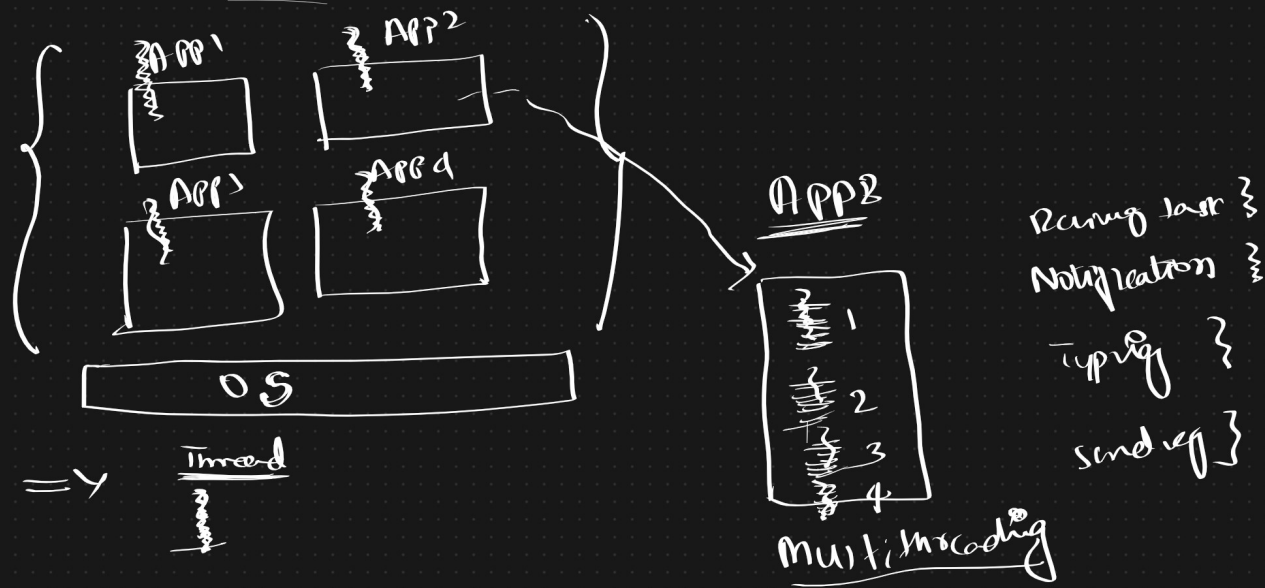
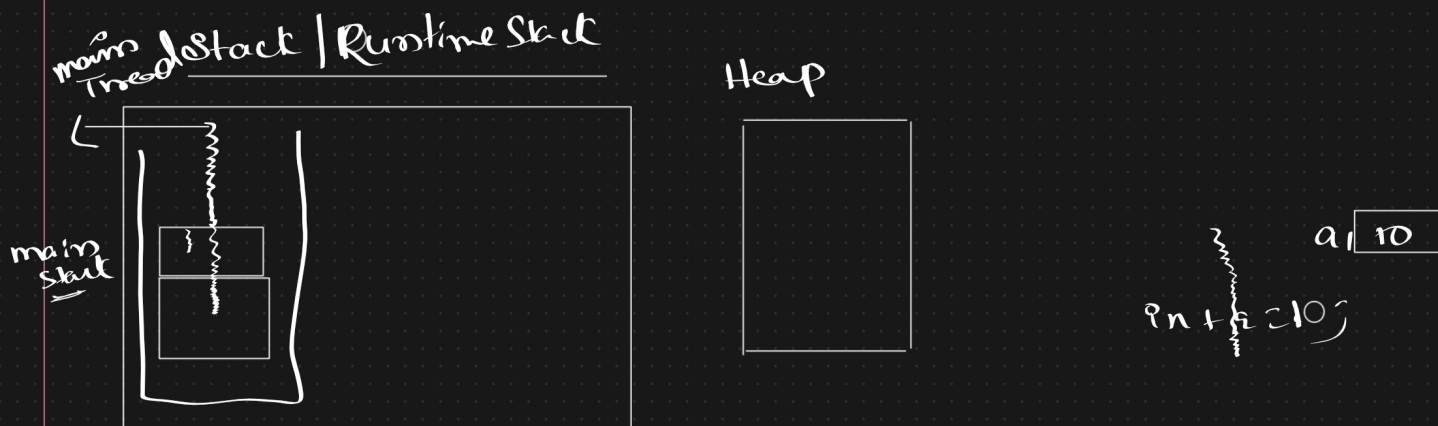


=> Threads :-



=> Thread

Thread -> line of execution



=> Thread => ✓

Problem of single threaded programming => ✓

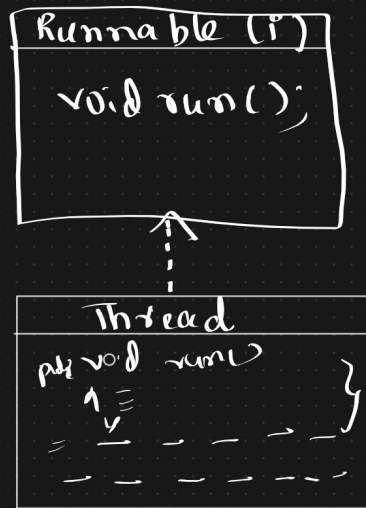
Need multithreads -> ✓

=> How do we create multiple threads?

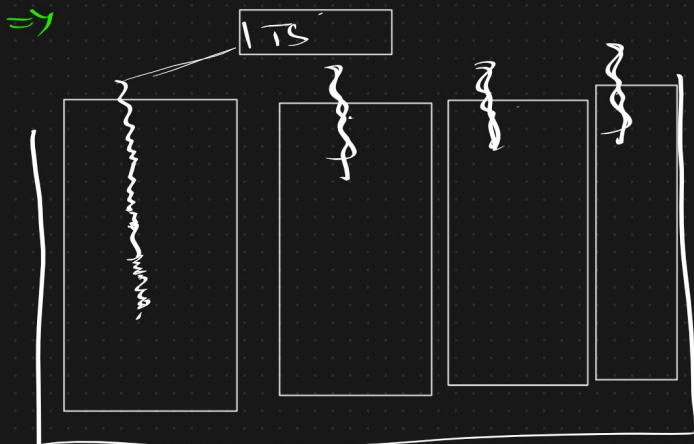
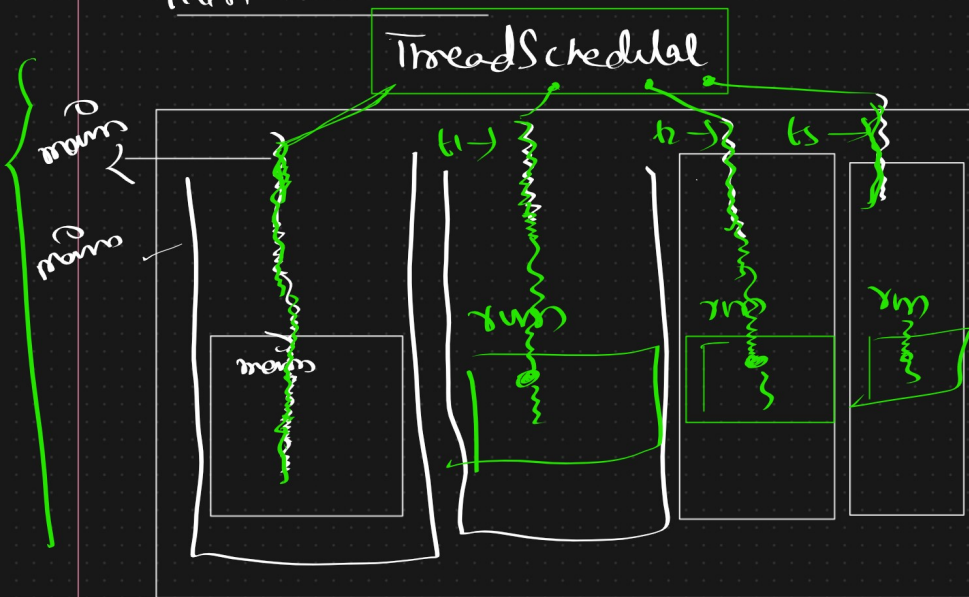
=> To deal threads => Thread

=> 2 ways to create threads :-

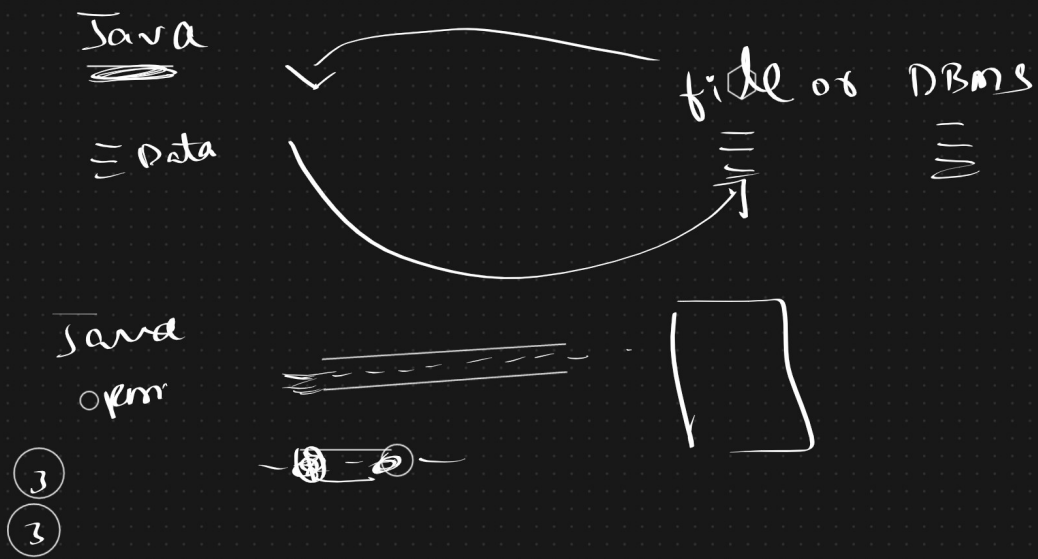
- ① extending Thread
- ② implementing Runnable(i) }



Runtime stack.



Search @hmaracollege on telegram



The diagram compares the performance of `StringBuffer` and `StringBuilder` in Java. It is divided into two main sections: `StringBuffer` and `StringBuilder`.

StringBuffer:

- Operations: `append()`, `insert()`, `delete()`, `replace()`, `reverse()`, `toString()`, `length()`, `capacity()`, `trimToSize()`.
- Thread Safety: Indicated by a large 'S' and the text 'Synchronized' with a checkmark, meaning it is thread-safe.
- Performance: Indicated by a large 'X' and the text 'Slow', meaning it is slower due to synchronization.

StringBuilder:

- Operations: `append()`, `insert()`, `delete()`, `replace()`, `reverse()`, `toString()`, `length()`, `capacity()`, `trimToSize()`.
- Thread Safety: Indicated by a large 'S' and the text 'Not Synchronized' with a checkmark, meaning it is not thread-safe.
- Performance: Indicated by a large 'X' and the text 'Fast', meaning it is faster as it does not have synchronization overhead.

A large bracket on the right side groups both classes under the heading 'String Builder'.